第6章 建模与UML

◆模型

- 是具体事物的抽象，建立模型的目的是在建造一个事物之前来理解它。
- 模型忽略了非本质的细节，因此模型就更容易让人理解。
- 通过建模，可以在创建一个复杂的系统之前，首先抽象出具体事物的不同视图，使其准确的表示设计方案，然后再验证模型是否满足需求，并逐步完善模型的细节，最后将模型转换为最终的实现

◆模型的主要用途有：

- ●在建造物理实体之前先验证
  - ●通过计算机软件来仿真和模拟事物，来理解事物的物理结构和运行特征可以通过
- ●与客户交流
  - ●通过模型为用户提供更好的感性认识，提高了客户对产品的认知，从而加强彼此之间的交流和沟通
- ●可视化
  - ●通过建立模型，可以给出一种真实的视觉效果，从而展示事物的组织结构和工作机理
- ●降低复杂度
  - ●把一些重要的结构和过程抽象出来，去掉一些无用的、影响系统理解的部分，从而降低系统的复杂度

- 抽象的核心在于根据系统的需要来提取事物重要方面，并抛弃那些不重要的方面。
- 抽象是不完整的，也是不准确的。抽象的意图是确定事物的领域，即系统的边界，从而更好地理解真实的系统。
- 对于一个事物的抽象，没有完全“正确”的模型，只有充分和不充分的模型。

6.1.3 面向对象建模

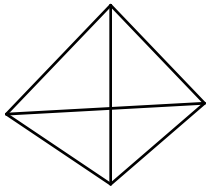
- 使用真实世界的概念模型来思考问题的一个方法，面向对象建模的最基本的元素是对象，对象既包含数据结构，又包含行为，对象之间可以进行消息通信。
- 面向对象的概念
  - 面向对象（Object Oriented）把软件系统变成了一系列离散的、合并了数据结构和行为的对象
  - 面向对象方法中，包含4个方面的特征：标识（identity）、分类（classification）、继承（inheritance）和多态（polymorphism）

- 标识指的是数据被量化为对象（object），它是离散的、可辨识的实体。

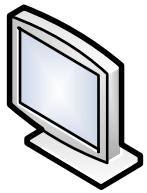
一张二维表、一个图结构、一个监视器、张三的汽车、李四的汽车以及一个求最大值算法都是对象。每个对象都有其自己的内部标识，而且是唯一的，即使对象的所有属性的值都相同，两个对象也是有差别的

变量名	变量值
信用卡号	6226010231198
账户	62260112571078825

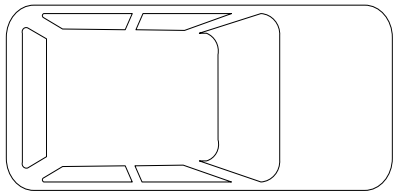
一张二维表



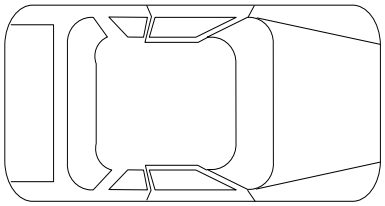
一个图



一个监视器



张三的汽车



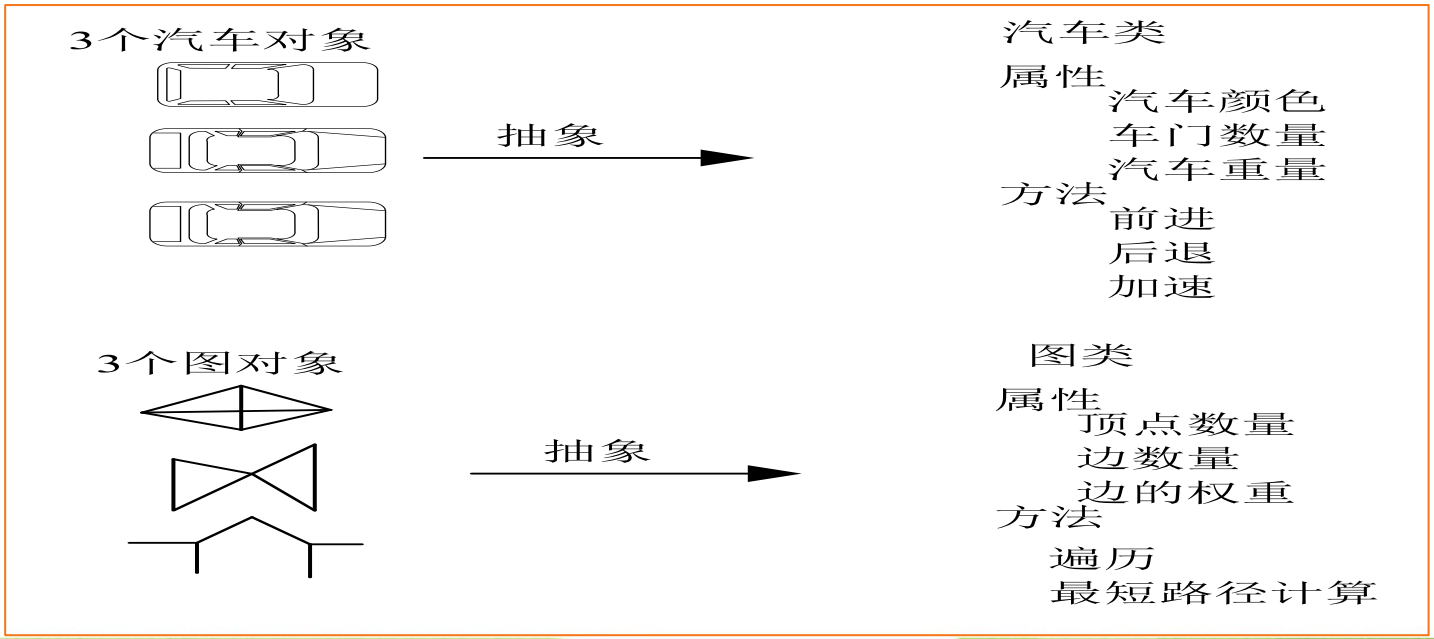
李四的汽车

```
for(int i=1;i<arr.length;i++){
   if(arr[i]>max){
          max=arr[i];
      }
}
```

求最大值算法

6.1.3 面向对象建模（续）

- 分类指的是有着相同数据结构（属性）和行为（方法）的对象被抽象为一类。
  - 监视器、二维表都是类的表述。类（class）是一种抽象，它从问题域的角度出发描述了其必须具有的属相，忽略了问题域不相关的属性。类的任何选择都与用户的问题域相关。
  - 每个类可能由很多独立的对象组成，每个独立的对象是该类的一个实例（instance）。对于属性，每个对象都有自己的取值，但是它与其它对象都拥有相同的属性和行为。图6.2显示了两个类以及它们包含的对象



6.1.3 面向对象建模（续）

- 继承指的是多个类的一种分层关系，类之间存在属性和操作的共享。子类拥有父类的属性和操作，即称子类继承父类的特征。
- 例如中学生和大学生是学生的子类，两个类都继承学生类的特征。不管是中学生还是大学生，都需要上课、考试等，但是它们又具有各自独有的特征，如大学生有学分，而中学生有会考等。
- 把几个类的公共特征提取出来而形成父类，这种设计方式减少了建模中的重复

6.1.3 面向对象建模（续）

- 多态是指对于不同的类来说，相同的消息会产生不同的动作。
- 例如，对于考试而言，大学生的课程考试和中学生考试不同。由特定类实现的操作被称为方法。因为面向对象操作符是多态的，所以它可能包含多种实现内容，对于不同对象的类来说，每一个都会有一个实现方法。

6.1.3 面向对象建模（续）

- 多态是指对于不同的类来说，相同的消息会产生不同的动作。例如，对于考试而言，大学生的课程考试和中学生考试不同。由特定类实现的操作被称为方法。因为面向对象操作符是多态的，所以它可能包含多种实现内容，对于不同对象的类来说，每一个都会有一个实现方法。
  - 在面向对象编程语言中，当向某个操作对象发送一个操作名时，收到消息的对象根据自己的类的不同，自动选择正确的方法来执行。对应用户来说，不需要知道有多少种方法实现了给定的多态操作。当开发者要增加新的操作时，只要新添加类，为同样的操作名对应的方法提供不同的内容就可以

- 面向对象建模
  - 面向对象建模是面向对象开发过程中使用一种图形化的表现方法来描述面向对象概念。在软件开发过程中，用来构建开发过程中不同阶段的模型。从分析设计到实现，使用相同的表示方法来建立模型。所以面向对象建模分为：需求分析、系统分析、系统设计、类的设计与实现。
  - 需求分析存在于软件开发初期，目的是从概念世界的角度阐述系统的要求。
  - 系统分析是一种简明准确的抽象，它从计算机世界的角度描述目标系统要做什么，而不是说明如何做。

- 面向对象建模
  - 系统设计用于解决应用程序的问题，系统设计人员需要确定优化哪些特性，选择何种策越来解决问题，完成哪些临时性资源分配等。
  - 类的设计是为分析模型添加具体的细节，其主要功能是为每个类设计数据结构和算法。
  - 实现是将类设计的成果转换为某种编程语言的过程。

6.2 UML简介

- 什么是UML?
  - 统一建模语言（Unified Modelling Language）的简称。
  - 是面向对象系统分析设计时常用的一种标准标记方法。
  - 由OMG(Object Management Group)软件标准化组织进行统一标准化的。
  - 仅仅只是一种描述语言，规定了一些表示方法，和具体的开发语言以及开发过程没有任何关系。

6.2.1 UML概述

- UML可以参与软件开发的各个阶段，即从软件开发的领域分析、需求分析、概要设计、详细设计、测试以及到软件后期的运行和维护的整个阶段。
- UML不是一门程序设计语言。但可以使用代码生成器工具将UML模型转换为多种程序设计语言代码，或使用反向生成器工具将程序源代码转换为UML。
- UML是一种离散的建模语言，不适合对诸如工程和物理学领域中的连续系统建模。它是一个综合的通用建模语言，适合对诸如由计算机软件、分布式计算系统或大数据处理过程的建模。

6.2.2 UML发展过程

- 面向对象的开发方法
  - 80年代初，随着Smalltalk语言的广泛使用，掀起了一场“面向对象运动”。随后，C++、Eiffel和CLOS等面向对象编程语言诞生了。
- 统一工作
  - 1996年，软件标准化组织OMG（Object Management Group）出面，制订一套符合OMG要求的方法，并设计出一种统一的建模语言，1997年9月，被合并成一套UML方法并提交到OMG。
- 标准化
  - 1997年11月，UML被OMG全体成员一致通过，并被采纳为标准。
- 目前最新的标准是UML 2.5.1, 发布于2017年12月。

6.2.2 UML发展过程（续）

- 统一的意义
  - 各种表示法的统一
  - 软件开发全生命期的统一
  - 应用领域建模的统一
  - 编程语言和开发平台的统一
  - 不同开发过程的统一
  - 内部概念的统一

6.2.3 UML的目标

- 开发UML语言的目标有
  - 首先，最重要的目标是使UML成为一个通用的建模语言，可供所有建模者使用。它并非某人专有，且建立在计算机界普遍认同的基础上，即它包括了各种主要的方法并可作为它们的建模语言。
  - 其次，采用源自OMT Booch、Objectory及其他主要方法的表示法能够成为主流，即让它能够很好地支持设计工作，像封装、分块、记录模型构造思路等。
  - 最后，让UML能够准确表达当前软件开发中的热点问题，比如大规模、分布、并发和团体开发等。

6.3 UML与建模

- ◆6.3.1 UML的建模模型
- ◆6.3.2 UML建模概念域
- ◆6.3.3 模型与UML中的图形

6.3.1 UML建模模型

- 建模目的划分为：
  - 业务建模时调查企业的业务流程，以模型化的方式记录下来。
  - 需求分析建模，即分析业务以捕获需求，界定软件开发的范围，获得需要开发的系统的需求。
  - 分析建模，即从软件的角度，从需求中提取出所需要的类、类的属性方法，以及类之间的关系。
  - 设计建模，即从实现的角度，加入平台、框架、实现方式、设计模式等实现方面的需求，修改并完善模型以满足这些需求。

6.3.1 UML建模模型

- 建模的内容划分为：
  - 结构建模对系统静态构造进行建模，即使用类图、对象图来描述系统的内部构造，使用构件图、部署图来描述系统的物理构造。静态结构定义了系统中的重要对象的属性和操作以及这些对象之间的相互关系。
  - 行为建模，即对系统中的行为、交互进行建模，使用交互图、状态图等来描述系统内部的动作。

6.3.2 UML建模概念域

- 静态结构
  - 确定有关应用、内部特性及其相互关系的关键概念
- 动态行为
  - 一种是根据一个对象与外界发生关系的生命历史；
  - 另一种是一系列相关对象之间当它们相互作用实现行为时的通信方式
- 实现构造
  - UML模型既可用于逻辑分析又可用于物理实现。
  - 一些模型代表设计，如类图、对象图等，一些组件代表实现，如部署图等

6.3.2 UML建模概念域

- 模型组织
  - 包是UML模型通用的层次组织单元，它们可以用于存储、访问控制、置以管理配及构造包含可重用的模型单元库。
- 扩展机制
  - 构造型加上了一些附加限制，具有新的解释和图标。
  - 标记值是能够被连接到任何一种模型元素上并代表任何信息
  - 约束是用某种特定语言的文本字符串表达的条件专用语言

6.3.3 模型与UML中的图形

- UML1.5中定义了9种图形，UML2.0中又追加定义了4种图形，共13种图形。
- 每种图形在不同的工程阶段，从不同的侧面对系统进行表示。
- 并不一定要使用所有的图形，根据需求选择合适的图形进行绘制。
- 各种图形的特征如下：

6.3.3 模型与UML中的图形

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

类型	图名		概要说明
静态结构图	类图		**表示分析**设计领域的静态结构。
	对象图		**表示系**统中的多个对象在某一时刻的状态。
	包图		对UML的模型元素进行分组整理，还可**表示多个包之**间的关系。
	构件图		**表示构成**软件的各个可重复再利用的部件的内部结构以及部件之间的相互关系。
	部署图		**表示系**统执行时的硬件构成以及硬件节点之间的关系。
动态行为图	用例图		**表示从系**统外部看到的系统所提供给用户可使用的功能。
	活动图		**表示某个业务的具体**处理流程，或者程序的某个算法。
	交互图	顺序图（时序图）	**按照**时间顺序从上到下来表示生命线之间的消息交互。可结合用例图用来表示某个用例的脚本。
		通信图（协作图）	**以参与交互的生命线之间的连接**为着眼点来表示生命线之间的消息交互。
	状态图		表示某个对象在其生命周期内，各个状态之间的迁移变化以及引起状态迁移的触发事件之间的关系。

6.3.3 模型与UML中的图形

- 从静态结构、动态行为和对象之间的交云角度来建模系统的不同方面，把模型划分为类模型、状态模型和交互模型。
- 每种模型捕获了系统重要的一个层面，但完整描述就需要全部三种模型。
- 类模型表示系统静态的、结构化的“数据冶层面;
- 状态模型表示系统时序的、行为的“控制冶层面;
- 交互模型表示各个对象的协作，是系统的“交互冶层面。
- 一般的软件过程具备所有这三个方面：它使用数据结构(类模型)，按时间设定操作顺序(状态模型)，并在对象之间传递数据和控制(交互模型)。每种模型都包含了对其他模型中的实体的引用。例如，类模型将操作依附于类，而状态模型和交互模型则详细描述这些操作。

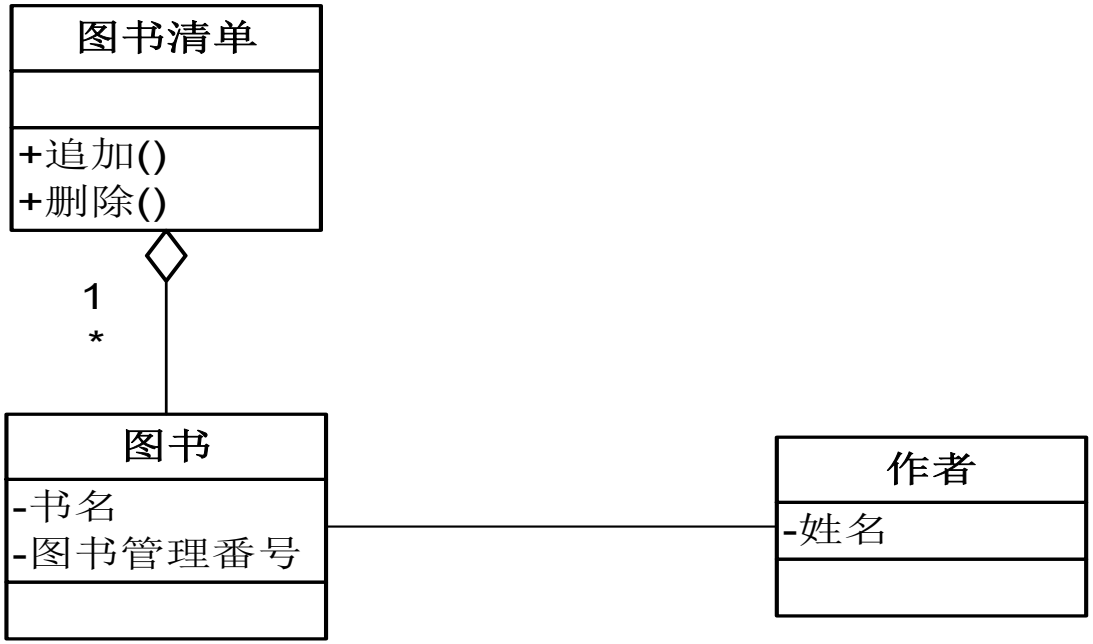
类模型

- 类模型描述系统中对象的结构，即它们的标识、与其它对象的关系、属性和操作。类模型提供了状态和交互模型的上下文。
- 在构建类模型的过程中，目标是从真实世界中捕获那些对应用而言重要的概念。类图表达了类模型。泛化使得类可以共享结构和行为，关联使得类之间发生关系。类定义了每个对象所携带的属性值，以及每个对象执行或经历的操作。

类模型-类图

- 类就是问题域的静态概念。
- 类由类名，属性和操作三部分组成。
- 类图用来描述系统的静态结构。

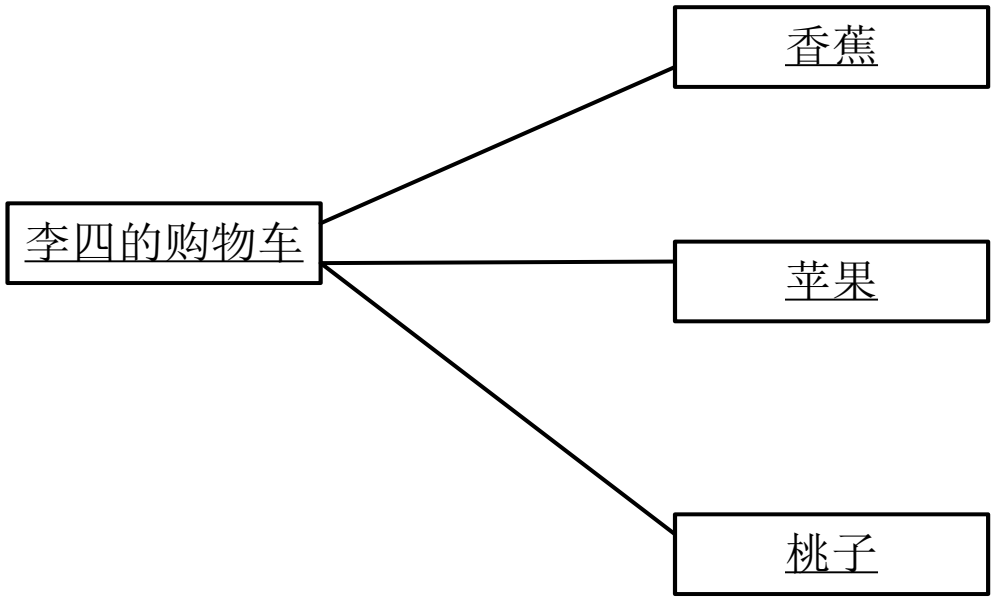
图书管理系统的类图：



对象图（Object Diagram）

- 对象图描述在某一时刻系统内对象的状态。
- 可用来辅助决定类图中类和类之间的多重度关系。

购物系统的对象图：



状态模型

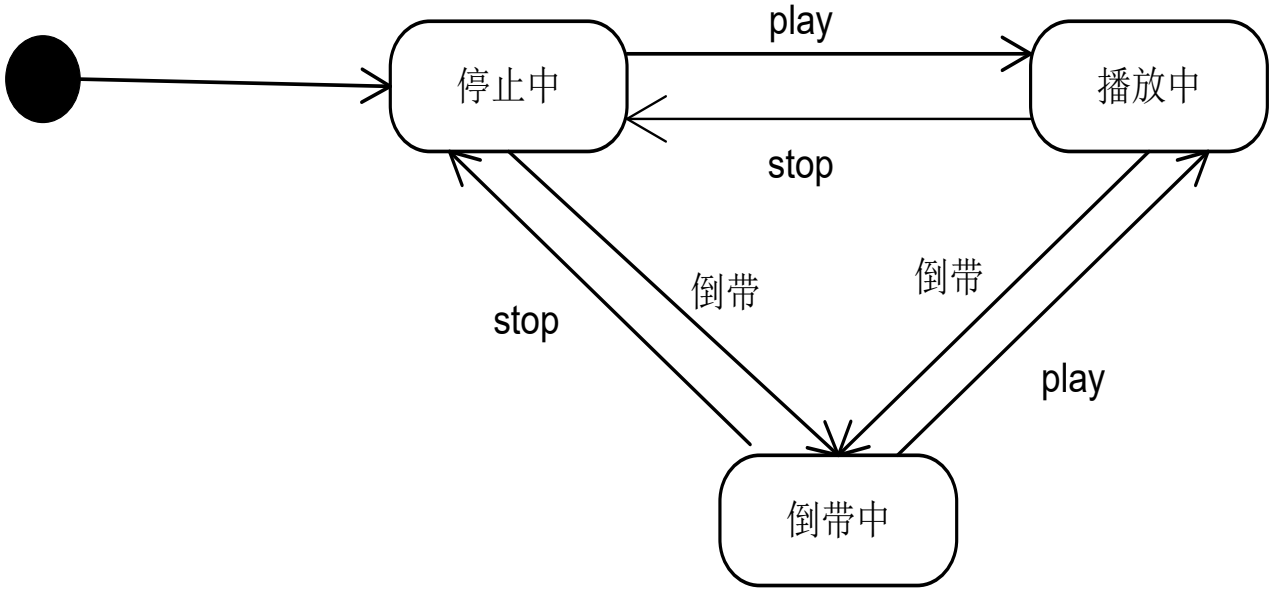
- 状态模型描述了与操作的时间和顺序相关的对象层面，即标记变化的事件、界定事件上下文的状态，以及事件和状态的组织。状态模型捕获控制，也即描述操作出现顺序的系统层面，不考虑操作作了什么，它们在操作什么，或者它们是如何实现的。
- 状态图表示状态模型。每幅状态图都显示了系统内允许的某个对象类的状态和事件序列。状态图会引用其他模型，状态图中的动作和事件类模型中对象上的操作。状态图之间的引用变成了交互图中的交互。

状态模型-状态图（Statemachine Diagram）

- 描述某个对象在生命周期内随着外部事件的触发而进行的状态变化的图。

  -着眼于一个对象，描述该对象在生命周期内的状态迁移变化情况。

- 状态图的组成部分
  - 状态（初始状态，终了状态）
  - 事件



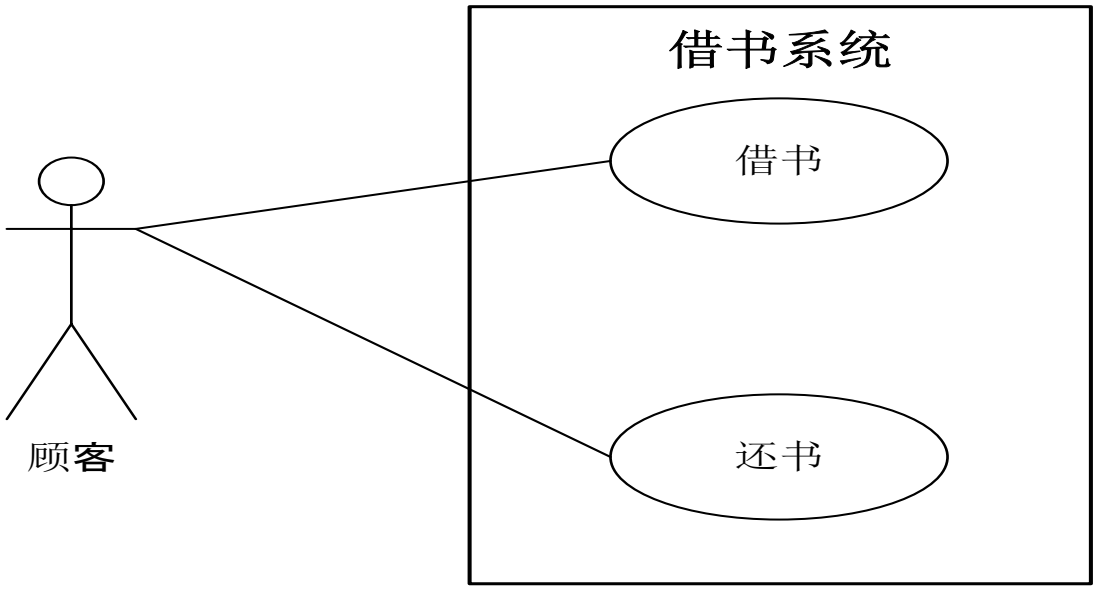
交互模型

- 交互模型描述了对象之间的交互，即独立对象如何协作，从整体上完成系统的行为。状态和交互模型描述了行为的不同侧面，它们两者结合才能完整描述行为。
- 用例图、顺序图和活动图评述交互模型。用例评述系统和外部参与者之间交互的主要内容。顺序图显示交互的对象和交互的时间顺序。活动图显示计算的处理步骤之间的控制流。

交互模型-用例图（usecase diagram）

- 表示的是从外部看到的系统所具有的功能。
- 主要在需求分析阶段使用。根据从用户收集来的需求，明确开发对象时使用。
- 组成要素：
  - 参与者（actor）
  - 用例（usecase）
  - 系统边界（可选）

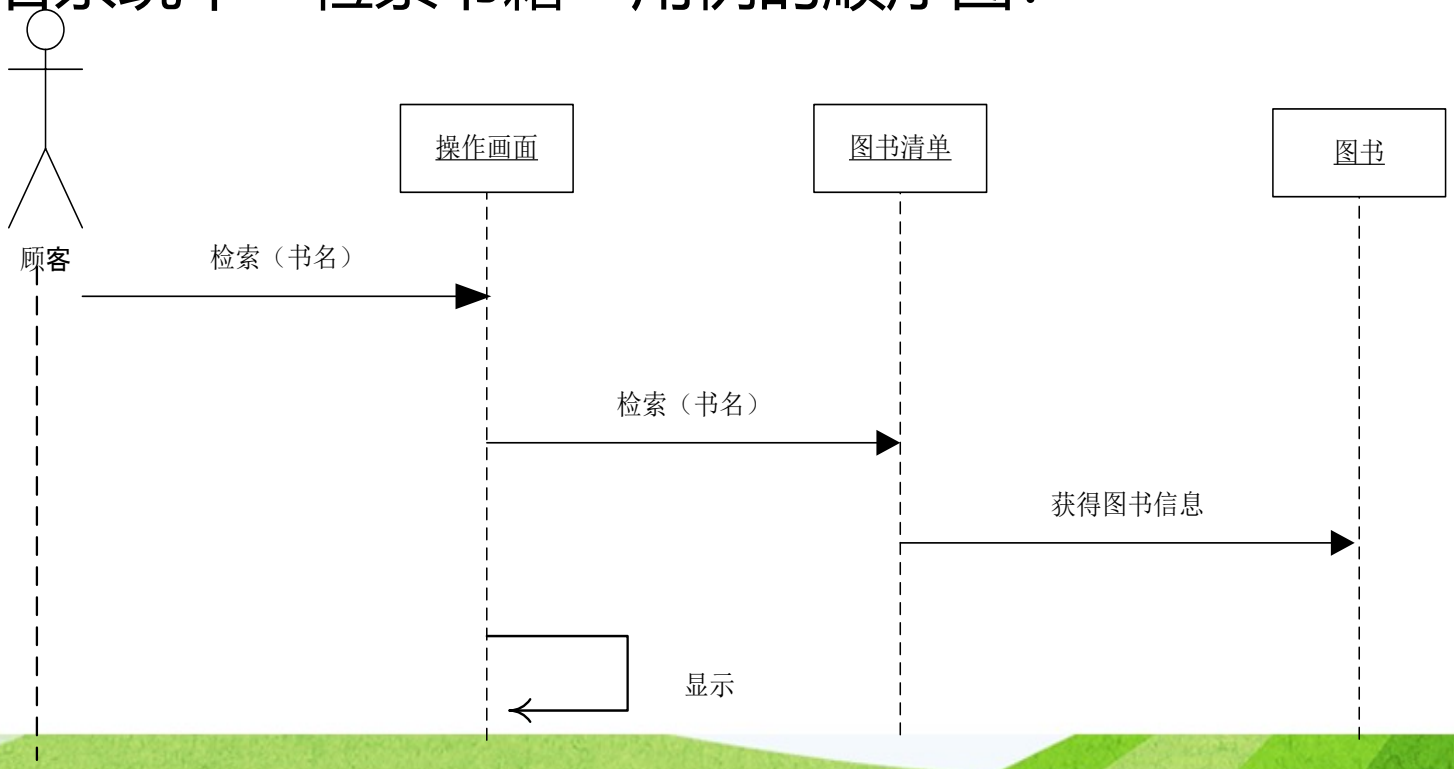
顾客利用系统借书的用例图：



交互模型-顺序图（Sequence Diagram）

- 交互图的一种，描述系统的动态行为
  - 按照时间顺序来描述生命线（对象，组件实例）之间的消息交换
- 顺序图的组成部分
  - 生命线
  - 消息交互

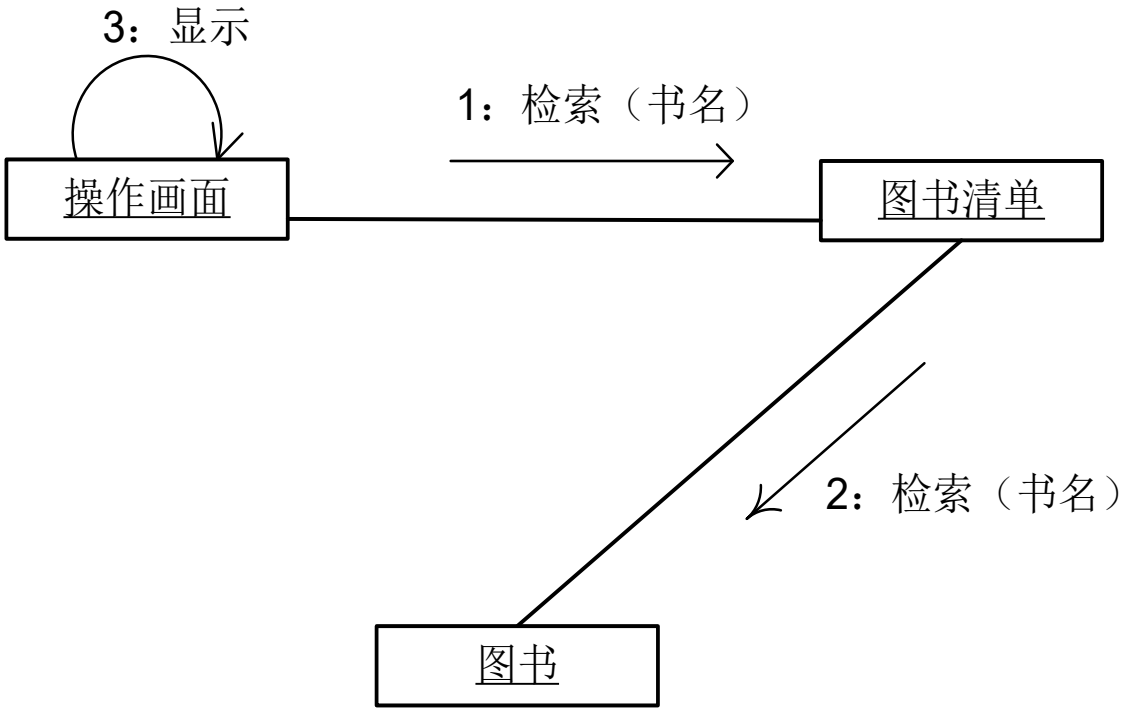
网上书店系统中“检索书籍”用例的顺序图：



交互模型-通信图（Communication Diagram）

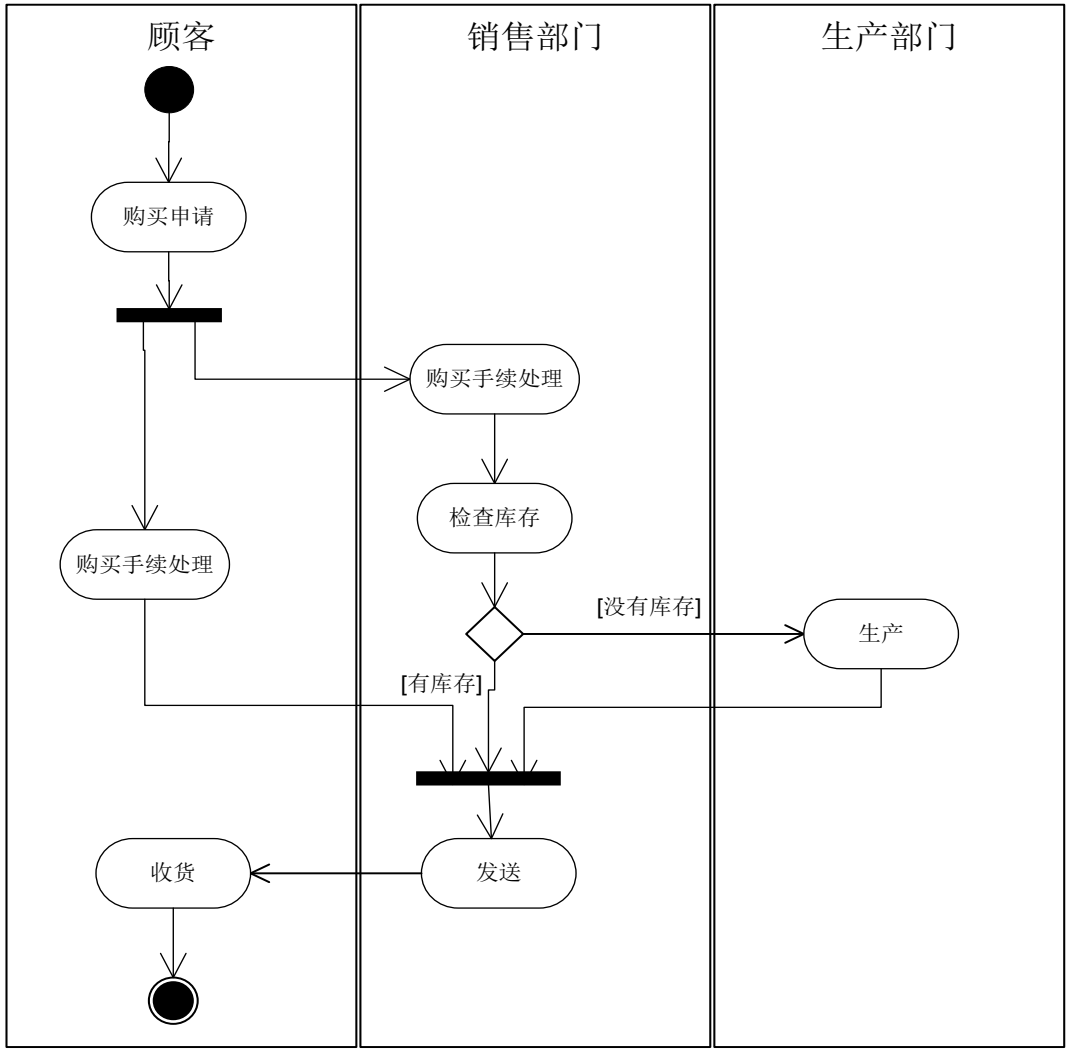
- 交互图的一种，用来描述系统的动态行为
  - 以生命线之间的连接关系为中心来描述消息交换的。
- 通信图的组成部分
  - 生命线
  - 消息

网上书店系统中“检索书籍”用例的通信图：



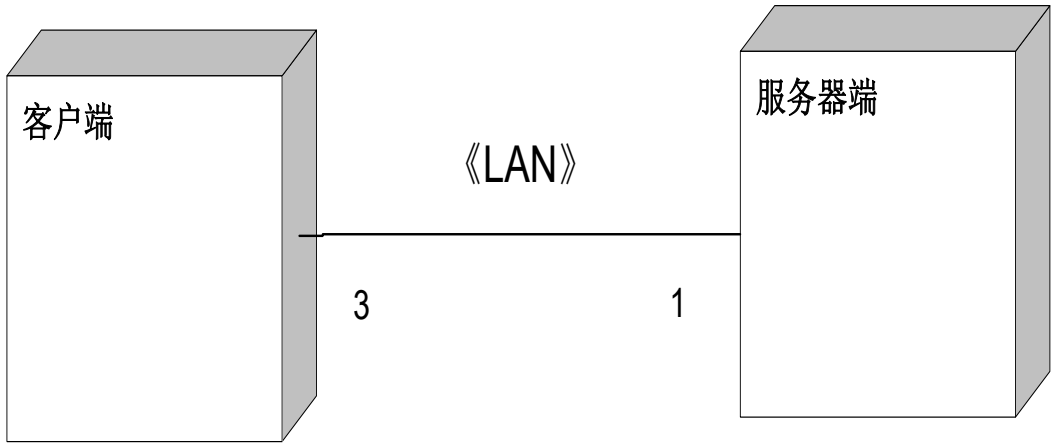
交互模型-活动图（Activity Diagram）

- 描述系统业务流程，用例流程的图
- 活动图的组成部分
  - 开始节点，终了节点
  - 活动
  - 判断节点和监护条件
  - fork节点
  - join节点
  - 参与者



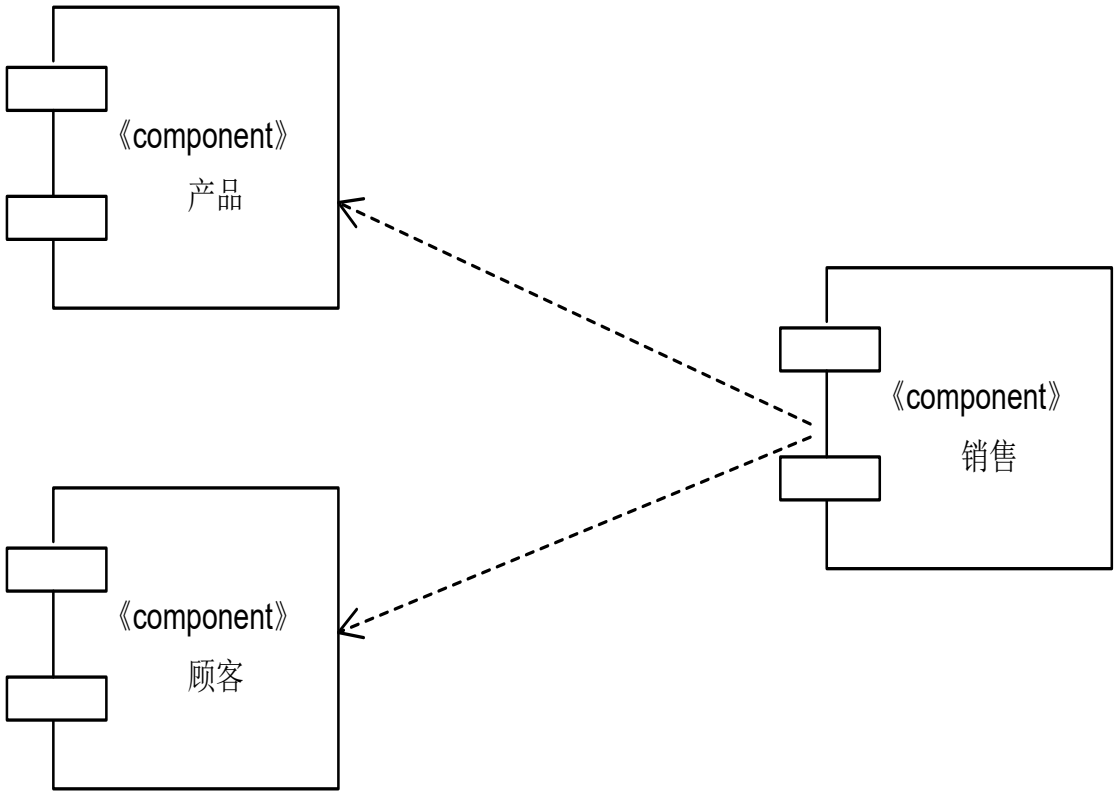
部署图（Deployment Diagram）

- 描述软件运行时的硬件，配置在硬件上的软件和硬件节点之间关系的图。
- 部署图的组成部分：
  - -节点
  - - 节点之间的关系



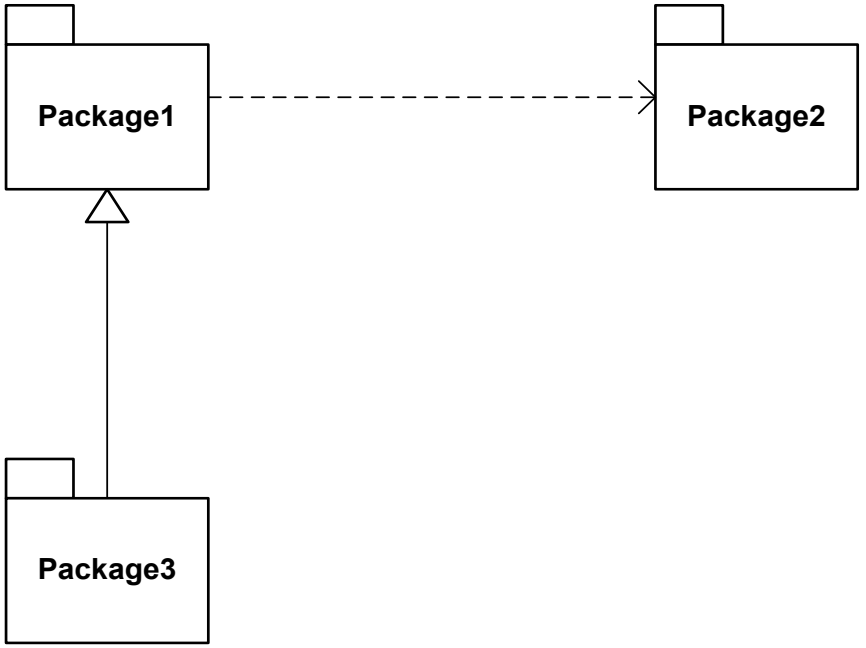
构件图（Component Diagram）

- 描述构成系统的软件构件的结构及其相互关系的图
  - 构件为构成软件系统的可以再利用的部品。
  - 在设计阶段使用
- 构件图的组成部分：
  - 构件
  - 构件之间的关系



包图（Package Diagram）

- 对构成系统的模型元素进行分组整理的图
- 包图的组成部分：
  -包（package）
  -包之间的关系



注：UML2.0新追加图形

1. 什么叫面向对象

◆面向对象技术是一种以对象为基础，以事件或消息来驱动对象执行处理的程序设计技术。

◆从程序设计方法上来讲，它是一种自下而上的程序设计方法，它不像面向过程程序设计那样一开始就需要使用一个主函数来概括出整个程序，面向对象程序设计往往从问题的一部分着手，一点一点地构建出整个程序。

2. 对象

- ◆对象（object）是面向对象（Object-Oriented，OO）系统的基本构造块，是一些相关的变量和方法的软件集。
- ◆可以利用对象来代表现实世界中的对象：可用一个飞行试驾程序来代表现实世界中的正在飞行的飞机，或者用机床数控程序来代表现实世界中运行的机床。
- ◆同样可以使用对象来定义抽象的概念，比如，点击按钮事件就是一个用在GUI窗口系统的公共对象，它可以代表用户点击程序界面中一个确定按钮动作。

3. 类

◆类（Class）是具有相同属性和操作的一组对象组合，也就是说，抽象模型中的“类”描述了一组相似对象的共同特征，为属于该类的全部对象提供了统一的抽象描述。

◆例如名为“仓库”的类被用于描述为仓库管理系统的一个对象。

◆类的定义要包含以下的要素

第一，定义该类对象的数据结构（属性的名称和类型）

第二，定义对象所要执行的操作，也就是类的对象要被调用执行哪些操作，以及对象要执行哪些操作，比如数据库操作等等。

4. 消息与事件

- 所谓消息（Message）是指描述事件发生的信息，是对象间相互联系和相互作用的方式。一个消息主要由5部分组成：消息的发送对象、消息的接收对象、消息传递方式、消息内容（参数）、消息的返回。传入的消息内容的目的有两个，一个是让接受请求的对象获取执行任务的相关信息，另一个是行为指令。
- 所谓事件通常是指一种由系统预先定义而由用户或系统发出的动作。事件作用于对象，对象识别事件并作出相应反应 。
- 对象通过对外提供的方法在系统中发挥自己的作用，当系统中的其它对象请求这个对象执行某个方法时，就向该对象发送一个消息，对象响应这个请求，完成指定的操作。程序的执行取决于事件发生的顺序，由顺序产生的消息来驱动程序的执行。

1. 抽象

◆抽象（Abstraction）是一种简化现实世界信息的方法。忽略事物中与当前目标无关的非本质特征，强调与当前事物相关的特性。通过抽象可以:

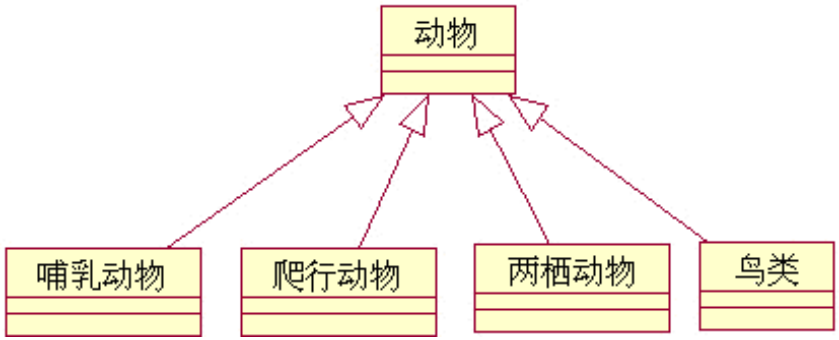
- ■将需要的事物进行简化。
- ■将事物特征进行概括。
- ■将抽象模型组织为层次结构。
- ■将软件重用得以保证。

2. 封装

- 封装（Encapsulation）是把对象的状态和行为绑到一起的机制，使对象形成一个独立的整体，并且尽可能的隐藏对象的内部细节。封装有两个含义：一是把对象的全部状态和行为结合一起，形成一个不可分割的整体。对象的私有属性只能够由对象的行为来修改和读取。二是尽可能隐蔽对象的内部细节，与外界的联系只能够通过外部接口来实现。
- 封装的信息屏蔽作用反映了事物的相对独立性，我们可以只关心它对外所提供的接口，即能够提供什么样的服务，而不用去关注其内部的细节问题。
- 比如说使用手机，我们关注的通常是这个手机能实现什么功能，而不太会去关心这个手机的是怎么一步步制造出来的。

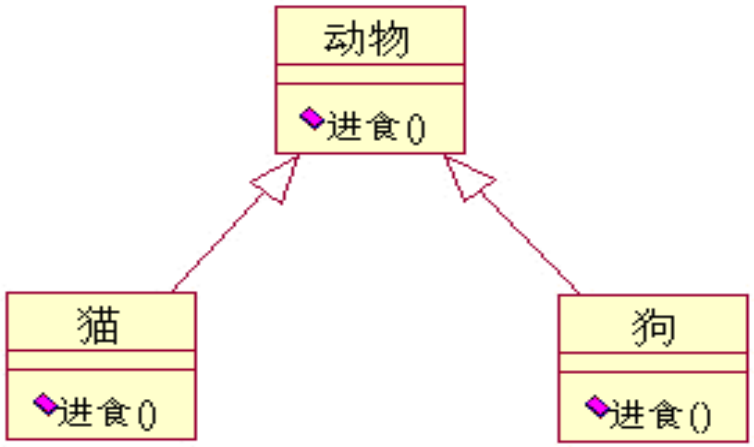
3. 继承

- 继承（Inheritance）是一种连接类与类之间的层次模型。继承是指特殊类的对象拥有其一般类的属性和行为。
- 继承意味着“自动地拥有”，即在特殊类中不必重新对已经在一般类中所定义过的属性和行为进行定义，而是特殊类自动地、隐含地拥有其一般类的属性和行为。
- 继承实现了对类的重用性，提供一种明确表述共性的方法。即一个特殊类既有自己定义的属性和行为，又有继承下来的属性和行为。



4. 多态

◆多态性（Polymorphism）是指在两个或多个属于不同类中同一函数名对应多个具有相似功能的不同函数，可以使用相同的调用方式来调用这些具有不同功能的同名函数。



模型

- 模型就是对现实客观世界的形状或者状态的抽象模拟和简化。模型提供了系统的骨架（sketch)和蓝图(Blueprint)。
- 建模是为了更好地理解系统。具体到软件开发来说：
  - 对于软件系统用户，展示开发人员对需求的理解。
  - 对于软件开发团队,模型有助于对软件需求和系统架构和功能进行沟通。
  - 对于软件的维护和技术支持者，有助于他们理解系统的架构和功能，迅速修复软件问题。

建模的目标和原则

◆目标

- ■便于开发人员展现系统
- ■允许开发人员制定系统的结构或行为
- ■提供知道开发人员构造系统的模板
- ■记录开发人员的决策

◆建模的基本原则

- ■认真地选择模型
- ■可以在不同精度级别上表示每一种模型
- ■最好的模型是与显示相联系的。
- ■孤立的模型是不完整的，尽可能从多视角建立模型。

补充： UML概述

- UML（Unified Modeling Language）的中文名称为“统一建模语言”，它是用来对软件密集系统进行可视化建模的一种语言；也是为面向对象开发系统的产品进行说明、可视化、构造和编制文档的一种标准语言。
- UML拥有足够的表达能力可以处理现代软件开发中出现的所有概念。
- UML是一个通用语言，不仅仅和一种通用程序设计语言一样，它也是一个庞大的标准符号体系，提供了多种的模型。

◆UML的主要特点

1. UML统一了Booch、OMT和OOSE等方法中的基本概念。
2. UML吸取了面向对象技术领域中其他流派的长处，其中也包括非OO方法的影响。
3. UML在演变过程中还提出了一些新的概念。

◆UML的应用领域

◆UML的目标是以面向对象图的方式来描述任何类型的系统，具有很宽的应用领域。其中最常用的是建立软件系统的模型，但它同样可以用于描述非软件领域的系统

◆UML适用于系统开发过程中从需求规格描述到系统完成后测试的不同阶段。

◆UML模型还可作为测试阶段的依据。

补充： UML的公共扩展机制

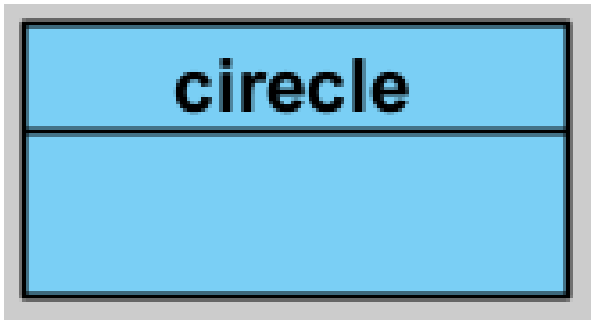
- UML 是一种通用的、工具支持的标准化建模语言，用来说明、可视化、构建和记录各种系统密集型过程(system intensive processes)的所有元素。
- UML 是定义良好的语言，但是：
  - 无法表达所有领域中所有模型的所有可能细微差别，因此……
  - 需要对UML进行扩展定制，或者裁剪UML以便交流。
  - 定制和裁剪必须以受控的方式进行
  - 解决方案：UML扩展机制(the UML Extension Mechanisms)

补充： UML的公共扩展机制

- 有四种可以在整个UML语言中一致使用的公共扩展机制
  - 1. 规格说明（Specifications）
  - 2. 修饰(Adornments)
  - 3. 通用划分(Common divisions)
  - 4. 可扩展性机制(Extensibility mechanisms)
- 其中最重要的是4。

1. 规格说明 (Specification)

- UML 不仅仅是一种图形语言。 相反，在其图形符号的每个部分背后都有一个文本的规格描述，描述该构建块的语法和语义。
- 例如，类图标背后是一个规格描述，它提供了该类的的全套属性、操作（包括其完整签名），体现类的所有型为；
- 从视觉上看，该类图标可能只显示该规格描述的一小部分。
- 该类可能有另一个视图，它呈现一组完全不同的部件，但仍然与该类的底层规格说明一致。
-



1. 规格说明(Specification)

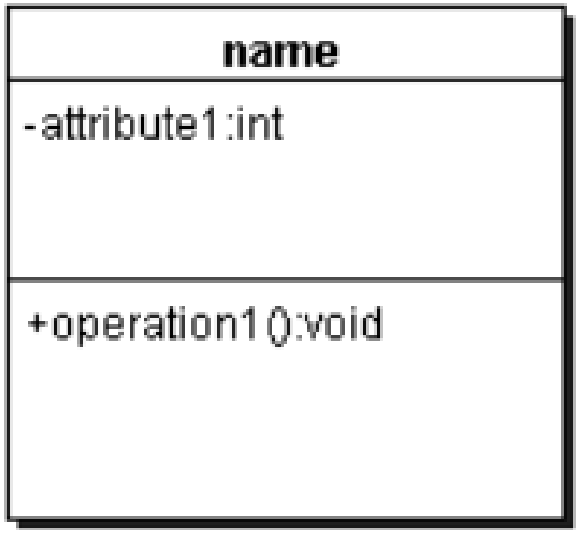
- 可以使用UML的图形符号来可视化系统；使用UML的规格说明来描述系统的细节。
- 鉴于这种划分，可以通过绘制图行符号然后向模型的规格说明添加语义来逐步构建模型。
- 或者直接通过创建规格说明（可能通过对现有系统进行逆向工程），然后创建投射到这些规格说明的图形符号来构建模型。
- UML 的规范提供了一个语义底层结构，其中包含系统所有模型的所有部分，每个部分以一致的方式相互关联

补充： UML的公共扩展机制

1. 规格说明(Specification)

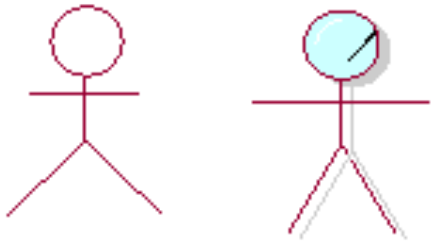
•因此，UML的图只是到该底层结构的视觉投影，每个图都揭示了系统的一个特定的方面。

• 例如右图包含 -attribute1:int 和 +operation1():void，它们是类name的规格说明。



2. 修饰(adornments)

- 在UML的图形表示中，每一个模型元素都有一个基本符号，这个基本符号可视化地表达了模型元素最重要的信息。
- 用户也可以把各种图形或者文本的修饰细节加到这个符号上以扩展其含义。这种添加修饰细节的做法可以为图中的模型元素在视觉效果上

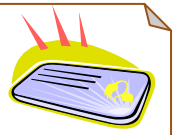


补充： 2. 修饰(adornments)

◆注释（Notes） - 最重要的修饰之一是注释，是用于呈现附加到元素或元素集合的约束或注释的图形符号；

Rendered as a rectangle with a dog-eared corner.

See smartCard.doc for details about this routine.



See http://www.rational.com for related info.

可能是文本和图形的组合.

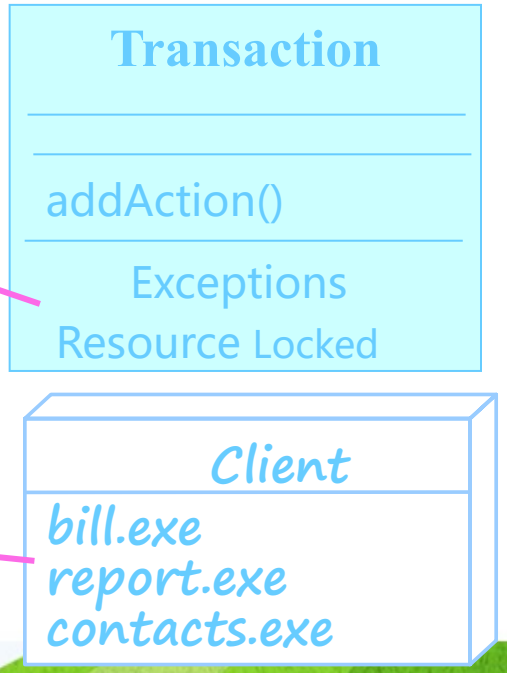
可能包括一个到外部链接的URL.

隔间(Compartments)：

- Placed near the element as
  - Text
  - Graphic
- Special compartments for adornments in
  - Classes
  - Components
  - Nodes

named compartment

anonymous compartment



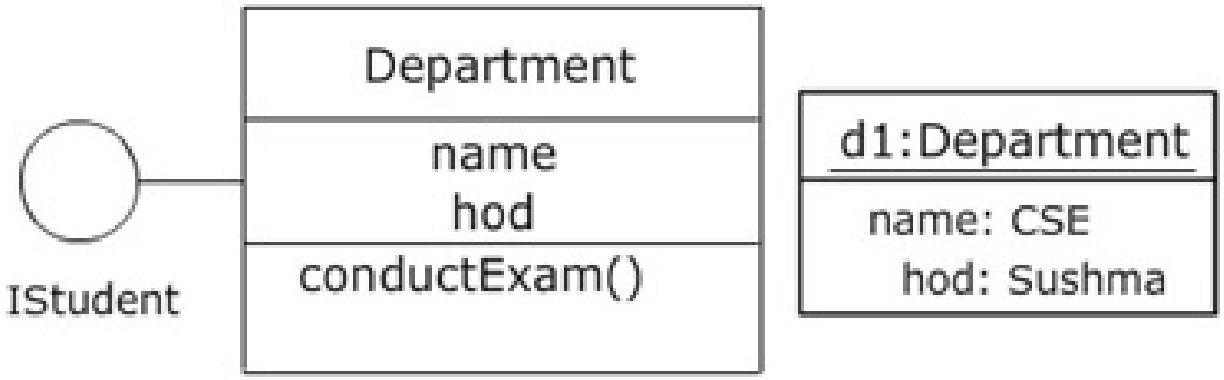
补充： 2. 修饰(adornments)

- 其他的修饰比如：
  - Multiplicity（多重性）
  - Roles（角色）：在类或对象之间的关系中，角色表示参与关系的一方可能扮演的角色。
  - Visibility（可见性）：指明属性、操作、接收信息的可见性，例如 + (公共), - (私有), # (受保护) 和 ~ (包内可见)。
  - Direction（方向）：特别用于消息和依赖关系中，指示信息流或依赖关系的方向，一般用箭头表示



3. 通用划分(Common divisions)

- 建模时可以从多个方式划分面向对象系统。
- 通常我们可以采用两种方式进行通用划分，一种是对类和对象的划分，另外一种是对接口和实现的分离。
- 对象和类使用同样的符号，对象名字带下划线。



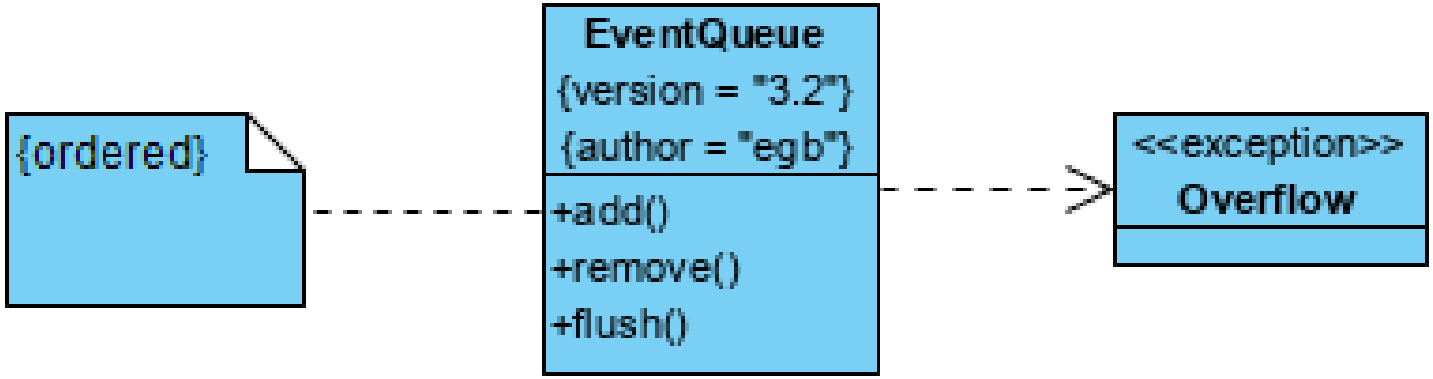
1. 构造型(Sterotype)

- 构造型扩展机制的目的就是基于一个已存在的模型元素再重新定义一个新的模型元素。
- 构造型的一般表现形式为使用“<<”和“>>”包含构造型的名称在里面，例如<<use>>、<<extends>>等。



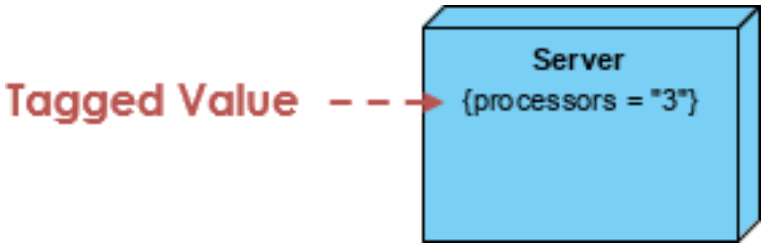
1. 构造型(Sterotype)

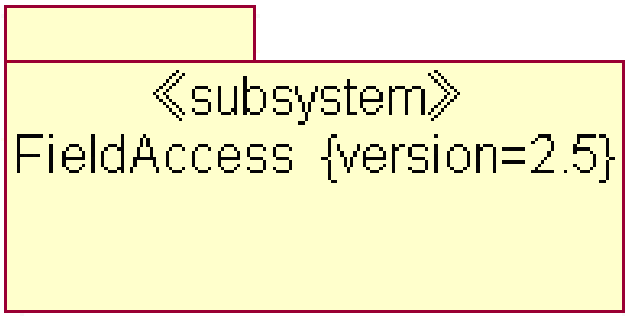




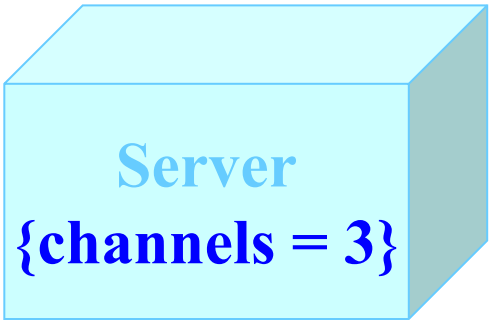
2. 标记值（Tagged value)

- 标记值是由一对字符串构成，这对字符串包含一个标记字符串和一个值字符串，用来存储着有关模型元素或表达元素的一些相关信息。
- 标记值，可以被我们用来扩展UML构造块的特性，可以根据我们的需要来创建新元素。
- 标记值可以与任何独立元素相关，包括模型元素和表达元素。标记值是当我们需要对一些特性进行记录时而给定元素的值。

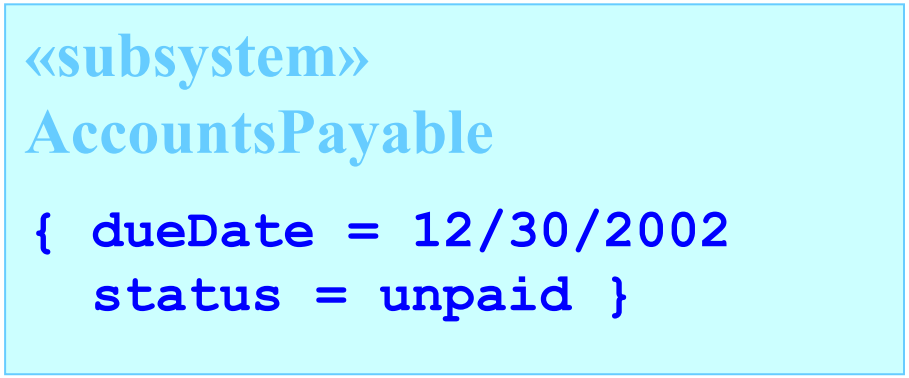




2. 标记值（Tagged value）







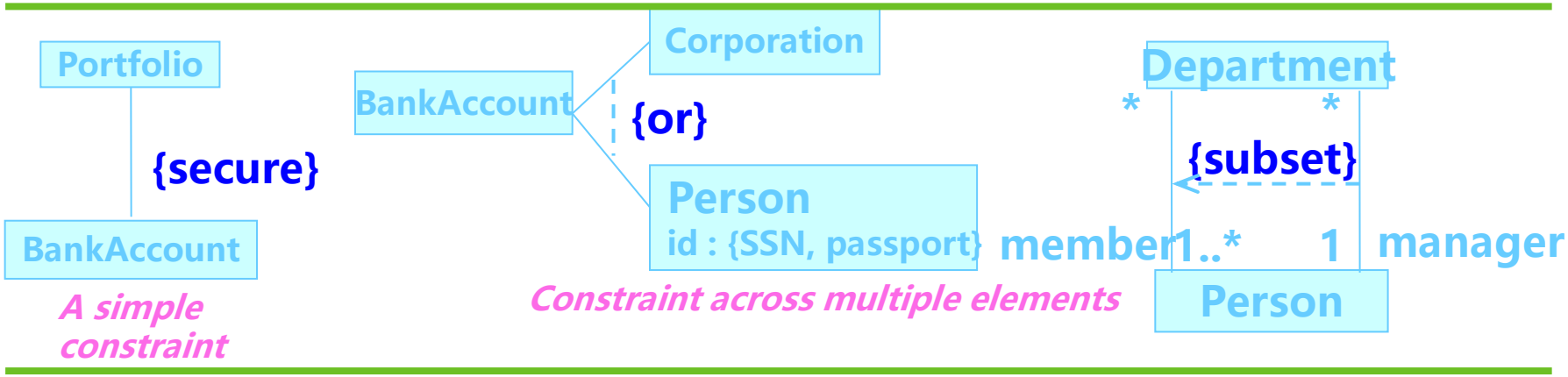
3. 约束(Constraint)

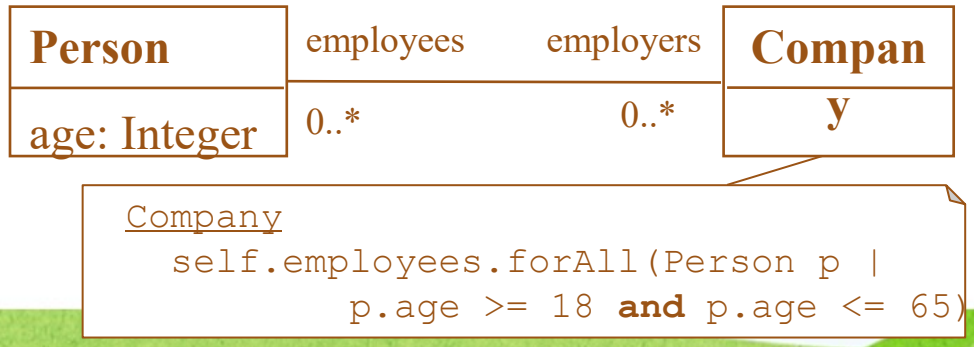
- 约束机制用于扩展UML构造块的语义，允许建模者和设计人员可以增加新的规则和修改现有的规则。
- 约束可以表示在 UML 的规范中不能表示的语义关系，特别是当陈述全局条件或影响许多元素的条件时，约束特别有用。
- 约束使用大括号和大括号内的字符串表达式表示，即约束的表现形式为{约束的内容}。约束可以附加在表元素、依赖关系，或注释上。



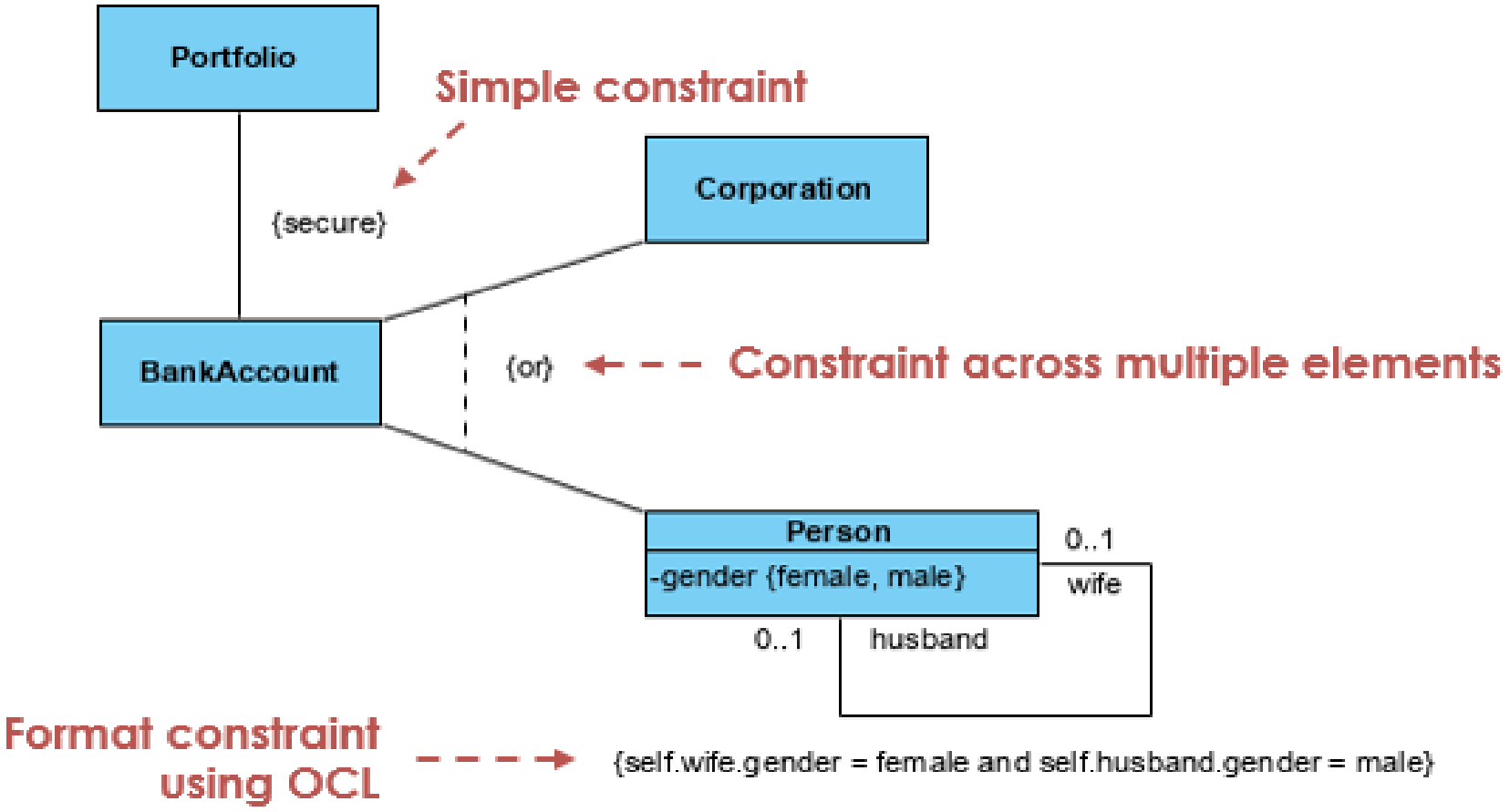
3. 约束(Constraint)

- Informally as free-form text, or
- Formally in UML's Object Constraint Language (OCL):
  E.g., {self.wife.gender = female and self.husband.gender = male}





3. 约束(Constraint)



补充：4. UML的可扩展性机制

- Profiles（UML 配置文件）：
  - Profile提供一种把一组相关的Stereotypes、Tagged values和Constraints打包的方式。一个Profile的specification会描述如何使用这些扩展元素来支持特定的建模领域或者建模语言的一个方言
- Profiles从UML 1.x就有。
- UML 2.0引入了UML配置图(UML Profile Diagram)，但UML配置图首次出现在 UML 图的“官方”分类中是在UML 2.2 规范中。

补充： UML工具

- ◆免费工具
  - ■推荐Visual Paradigm Community Edition 17
  - ■ 其次推荐BOUML 7.11
  - ■Eclipse Papyrus是一个相对复杂的系统。
    - □ Visual Paradigm Community Edition的不足是社区版没有逆向工程功能。
    - □作为逆向工程，BOUML比Eclipse Papyrus要操作简单些。
    - □因为Eclipse Papyrus需要安装对应插件，而且容易碰到版本兼容问题。另外BOUML的逆向结果优于Papyrus. Papyrus实现标准很完备，但是太庞大负责，操作性也不是很好。

补充： UML工具

- 商用工具
  - SAP Power Desinger 16
  - Enterprise Arechitecture 12
  - Rational Software Architecture 9
- 其他绘图工具或者软件
  - Visio
  - Draw.io
  - PlantUML
  - …